

GReinSS Tutorial — Speaker Notes

NCI Spring School on Algorithmic Cancer Biology

One block per slide. → NOTEBOOK slides are the live-demo hand-offs.

1. GReinSS

GReinSS tutorial — NCI Spring School on Algorithmic Cancer Biology Speaker notes are in HTML comments like this one. Live-demo hand-offs are marked “→ NOTEBOOK”.

Hi everyone. Today: a hands-on tutorial on GReinSS — a method for a problem that shows up all over algorithmic cancer biology: inferring hidden combinatorial states from noisy, indirect measurements. We’ll cover the idea, the one theorem that makes it work, and then train it live on your laptop. Goal: you leave able to apply it to your own problem.

2. A recurring statistical inference problem in computational biology

The unifying pattern: a hidden discrete structure S , indirect observation X , and a KNOWN or partially-known likelihood $\Pr(X|S)$. Trees, CNA sets, isoforms — all fit. This is “self-supervised”: the physics/biology of measurement is known; the state is not.

3. A learning and an inference problem

Panel a: hidden distribution over states S^* , each emits an X . We model $\Pr(S|\theta)$. Two problems: (1) LEARN θ from all observations jointly — the shared model couples them; (2) INFER the best state per observation. Everything today serves these two.

4. Why the usual tools struggle

EM: exact expectation needs summing over all states — only works for special structure (HMMs). VI: optimizes a lower bound (ELBO) instead of the true likelihood, and you must hand-design a tractable approximate posterior $q(S)$ over a combinatorial space — exactly what’s hard here. VAE: great generative models, but the latent lives in a made-up $\mathbb{R}^d$, not your isoform space. Local search: per-observation, no sharing of statistical strength. Naive PG / GFlowNet are the closest cousins to what we do — and we’ll see exactly why they fail.

5. The GReinSS idea

(no notes)

6. Primer on reinforcement learning (RL)

The generic RL mental model, deliberately provider-neutral and episodic: the policy builds an object action-by-action (a trajectory), a scalar reward scores the finished trajectory, and the goal is to maximize EXPECTED reward. Trace the three arrows aloud: sample τ from the policy $\rightarrow$ score it $\rightarrow$ policy-gradient nudge, then repeat. The one identity they must take away is REINFORCE: you can differentiate an expectation over samples by weighting each trajectory's log-prob gradient by its reward — no gradient through the reward itself. Everything on the next two content slides is this loop with a specific reward plugged in. Contrast up top with supervised learning to anchor the audience.

7. GReinSS: Generative Reinforcement Learning of Structured States

Key question: Can we adapt reward function $r(\tau)$ to optimize data likelihood $\Pr(X_{1:N} \mid \theta)$?

Same diagram, re-labeled — say it out loud: “nothing about the machinery changes.” Actions grow a discrete structure; the trajectory's terminal state IS the object we care about $S(\tau)$; the policy is a neural net; the reward (orange, highlighted) is the only novel piece and the objective is now the data log-likelihood, not a hand-picked reward. This is the pivot: GReinSS = policy gradient where the reward is engineered so that maximizing expected reward provably equals maximum-likelihood learning. Hold the suspense on the exact reward formula — that's the very next slide (the denominator is the whole trick).

8. Dynamically-changing rewards

The method in one line: what we optimize (the log-likelihood gradient) equals how we optimize it (a policy gradient with the dynamically rescaled reward). The numerator $\Pr(X_i \mid \tau)$ is “how well does this trajectory explain observation i ”; the DENOMINATOR $\Pr(X_i \mid \theta)$ is the model's total probability of X_i , which rescales each observation's contribution. Theorem 1: this policy gradient is unbiased for the log-likelihood gradient — gradient taken ONLY through $\log \Pr(\tau \mid \theta)$, the reward treated as constant each step.

9. GReinSS training loop

The training loop IS the RL cycle from the primer, with our reward plugged in: sample τ from the policy $\rightarrow$ score with $\Pr(X_i \mid \tau)$ $\rightarrow$ policy-gradient update θ . The one addition over vanilla RL is on the loop-back leg: the denominator $\Pr(X_i \mid \theta)$ shifts as θ learns, so each iteration we re-estimate it by sampling (average

$\Pr(X_i|\tau)$ over sampled trajectories). As a state gets covered its denominator grows and its reward shrinks — automatic load balancing. API surface: the user supplies only (a) a generator for S and (b) the likelihood $\Pr(X|S)$. Everything else — reward machinery, sampling, gradient — is provided. That's exactly what the notebook will show: write those two functions and call `train()`.

10. Off-policy learning (when on-policy is too slow)

Practical must-have for hard problems. Instead of blindly sampling from the policy, we tilt sampling toward states that actually fit each observation — provably the best proposal. In our biology applications this is where domain knowledge enters: a fast classical method proposes candidate states, and GReinSS refines the distribution over them. Keep this slide brief unless the audience asks.

11. Intuition behind dynamic rewards — why the denominator $\Pr(X_{imid} \theta)$?

Policy $\theta \equiv \Pr(\tau \mid \theta)$ over τ_1, τ_2, τ_3 (τ_j builds S_j); marginal $\Pr(X_i \mid \theta) = \sum_{\tau} \Pr(\tau \mid \theta) \Pr(X_i \mid \tau)$.

Reward **shrinks as it succeeds** □ the policy covers every observation.

θ IS the policy: the bars plot $\Pr(\tau|\theta)$, and each panel is the DIFFERENT θ^* that its reward selects. The values are exact optima, not eyeballed. Assume one X_1 and one X_2 . LEFT (raw reward = $\Pr(X_i|\tau)$): per-trajectory reward (.5,.3,.2); maximizing $E_{\tau}[r]$ is linear in the policy, so all mass goes to the top, $\tau_1 \rightarrow \theta=(1,0,0)$. Then $\Pr(X_2|\theta)=0$ and the joint $L=0$. RIGHT (rescaled): Thm 1 makes this maximize the data likelihood $L = \Pr(X_1|\theta) \cdot \Pr(X_2|\theta) = (.5 p_1)(.3 p_2 + .2 p_3)$. τ_3 is dominated by τ_2 for X_2 (.2<.3) so $p_3=0$; then $L = .15 p_1 p_2$ with $p_1+p_2=1$, maximized at $p_1=p_2=.5 \rightarrow \theta=(.5,.5,0)$, $L=.25 \times .15=.0375$ (the global optimum). Punchline: the denominator = automatic load-balancing across observations.

12. The scoreboard — who actually maximizes the likelihood?

The positive flip of the opening “why tools struggle” table. Same objective across the board; only GReinSS provably maximizes the joint data likelihood. Naive PG collapses to one state; GFlowNets match rewards, not likelihood; EM-based generative models approximate via point estimates; local search has no shared model. This is the one-slide “why us” scoreboard.

13. → NOTEBOOK · Demo 1: Set reconstruction

SWITCH TO JUPYTER. Walk through: load observations → define $\Pr(X|S)$ (one line) → build generator net → train ~200 epochs live (watch the likelihood

curve rise) → infer states → compare to naive thresholding. The punchline: GReinSS denoises using structure shared across observations, beating per-pixel rounding.

14. Results — simulations

Two combinatorial state types, same method. Left: graphs — GReinSS dominates especially when observations are information-poor (few walks). Right: sets — GReinSS is the only method that scales to large universes. GEM-based methods (VAE/autoregressive/diffusion) plateau; the closest RL cousins (naive PG, GFlowNet) fail. The reward rescaling is the difference.

15. → NOTEBOOK · Demo 2: Graph inference (pre-trained)

SWITCH TO JUPYTER (second section). Heavier model, so we ship a pre-trained checkpoint. Show: load model → simpleInference → compare predicted adjacency to the ground-truth graph we saved during pre-training → report F1 and visualize one graph. This mirrors the paper's Fig on graph inference but on your own generated instance with known truth.

16. Application — RNA isoforms beat RSEM

The payoff for this audience. Isoform quantification is a textbook latent-variable problem: short reads are indirect observations of full-length transcripts. RSEM is the standard EM tool GTEx ships. Dropping GReinSS in — with a trivial $\Pr(X|S)$ — matches long-read ground truth far better. Panel c: on MBD2, GReinSS recovers the two true isoforms with near-correct proportions; RSEM splits mass across wrong isoforms. Panel d: distribution of (GReinSS - RSEM) error is shifted negative → GReinSS wins across the genome.

17. GReinSS already powers two cancer methods

GReinSS isn't just a new paper method — it's the generalization of machinery that already produced two cancer-genomics tools. If you work on trees, CNAs, isoforms, or any grow-able discrete structure with a known likelihood, this framework likely applies to you.

18. When should you reach for GReinSS?

Decision guide. The two hard requirements: an incremental generator and a likelihood. If you have those, the four-line recipe is all you need to start — exactly what the notebook demonstrates.

19. Thank you — let's build

Wrap up: the method is one reward formula with a clean theorem, it beats the standard tools on simulations and on real isoform data, and it's a drop-in for discrete latent-state problems in cancer genomics. Open the notebook and try it on your own $\Pr(X|S)$.